VWO Login Dashboard Test Plan

Introduction

This test plan defines the robust verification strategy, scope, resources, and schedule for the VWO (Visual Website Optimizer) Login Dashboard (Target Production URL: app.vwo.com). Our primary objective is to validate that the authentication system securely and seamlessly supports B2B enterprise requirements, maintaining strict high availability (99.9% uptime) and exceptional performance (sub-2-second load times). The testing ensures a secure gateway for digital marketers, UX designers, and enterprise users relying on VWO's suite.

Scope (Tested & Not Tested)

In-Scope (Tested)

- **Authentication Flows:** Primary email/password verification, Multi-Factor Authentication (MFA/2FA) enforcement, Enterprise Single Sign-On (SAML, OAuth), and Social Login capabilities.
- **Input Validation & UX:** Real-time form field validation, proactive email formatting checks, password strength indicators, clickable labels, and comprehensive error handling.
- **Account Recovery:** Secure password reset flows and explicit password complexity requirements.
- **UI & Accessibility:** Responsive mobile-optimized layouts, dynamic light/dark mode themes, and strict WCAG 2.1 AA accessibility guidelines (ARIA labels, keyboard navigation, high contrast).
- **Security & Platform Integration:** SSL/HTTPS encryption enforcement, rate limiting mechanisms to deter brute-force attacks, and seamless transitions to the core VWO dashboard.

Out-of-Scope (Not Tested)

- **Internal VWO Platform Modules:** Post-authentication Analytics, A/B Testing experiment configuration, and core platform capabilities are excluded.
- **Third-Party Providers:** The internal logic and infrastructure of external Identity Providers (Google, Microsoft) and SSO services are excluded; only our integration points are covered.
- **Infrastructure:** Specific Backend server environments and Database testing layers, as the technical stack details are [NOT SPECIFIED].

Test Strategy/Approach

A pragmatic, systems-oriented hybrid approach will be implemented:

- **Automation First:** End-to-end UI functional and regression automation will be architected utilizing **Playwright**, chosen for its resilient cross-browser execution and modern web capabilities.
- **Functional Testing:** Thorough validation of the New User (signup routing) and Returning User (remember-me state) authentication journeys.
- **Security Testing:** Methodical application of OWASP security guidelines targeting authentication, session handling, and GDPR/CCPA data protection compliance. (Specific testing tool [NOT SPECIFIED]).
- **Performance Testing:** Concurrent load testing to simulate thousands of simultaneous requests to validate the 2-second load time metric. (Specific load testing tool [NOT SPECIFIED]).
- **Accessibility Testing:** Automated Axe scans integrated into Playwright, supplemented by manual screen-reader evaluations.

Test Environment

- **Automation Framework:** Playwright (Node.js/Python [NOT SPECIFIED]) executing against Chromium, WebKit, and Firefox engines.
- **Network Constraints:** Standard and throttled network conditions to benchmark the sub-2-second load constraint limits.
- **Target URLs:**
  - **Dev URL:** [NOT SPECIFIED]
  - **Staging URL:** [NOT SPECIFIED]
  - **Prod URL:** app.vwo.com

Test Criteria

Entry Criteria

- Front-end and Back-end development for the specific Phase is complete.
- Stable Dev/Staging environments are successfully provisioned and accessible. (Currently [NOT SPECIFIED]).
- Validated test data sets (valid users, invalid users, mocked SSO profiles) are injected.

Exit/Pass-Fail Criteria

- **Pass Criteria:**
  - Automated Playwright execution yields a 95%+ success rate on authentication attempts.
  - Page load performance remains strictly under the 2-second benchmark.
- **Fail Criteria:**
  - Identification of any critical or high-severity functional defects.
  - Occurrence of successful simulated brute-force intrusions or session hijacking incidents.
  - Failure to meet compliance audits (GDPR, CCPA, WCAG 2.1 AA).

Resources & Roles

- **Test Architect / Principal SDET:** Strategy formulation, framework creation, and architectural oversight.
- **Automation Engineers:** Playwright script engineering and CI/CD pipeline integration. (Resource Count [NOT SPECIFIED]).
- **Performance Analysts:** Load profile generation and real-time environment monitoring. (Resource Count [NOT SPECIFIED]).
- **Security Auditors:** Penetration testing and compliance validation. (Resource Count [NOT SPECIFIED]).

Schedule & Milestones

Testing activities are strictly aligned with the development milestone schedule:

- **Phase 1: Q1** - Validation of Core Authentication (Secure login forms, basic validations, session handling, and password resets).
- **Phase 2: Q2** - Validation of Enhanced UX (Mobile optimization controls, Accessibility features, advanced UI feedback).
- **Phase 3: Q3** - Validation of Enterprise Features (SSO integrations, advanced security, monitoring, and Social Login analytics).

Risk & Contingency

- **Risk 1:** Lack of explicit Environment configurations (Dev/Staging URLs).

    - *Mitigation:* Demand immediate infrastructure provisioning from DevOps or rely on mock servers/local hosts as temporary stop-gaps.
- **Risk 2:** Third-party Identity Provider disruptions during execution.
    - *Mitigation:* Implement rigorous API mocking and stubbing within the Playwright framework to isolate functional testing.
- **Risk 3:** Indeterminate Tech Stack (Frontend frameworks, API protocols).
    - *Mitigation:* Develop platform-agnostic automation scripts focused solely on DOM interaction and HTTP interception.

Deliverables

- Enterprise-grade Playwright Automation Repository & Framework.
- Traceability Matrices linking test scenarios to PRD constraints.
- Iterative Test Execution & Defect Coverage Reports.
- Final Accessibility (WCAG) and Security Audit Certifications.

🛑 Outstanding Doubts & Gaps Identified

Based on the provided contextual constraints, the following critical gaps require immediate clarification to finalize test readiness:

1. **Target Environments:** What are the explicit Dev and Staging URLs?
2. **Tech Stack Specifics:** Are there specific nuances in the Frontend (e.g., React vs. Vue) or API types (REST vs. GraphQL) that our automation framework must cater to for network interception?
3. **Tooling Decisions:** Which specific tools are approved for Security/Penetration testing and Performance/Load testing?
4. **Resourcing:** What is the allocated headcount budget for Automation, Performance, and Security engineers?
5. **Automation Language bindings:** Shall the Playwright framework utilize Node.js, Python, or Java bindings?